



CALIFORNIA STATE UNIVERSITY
FULLERTON

ISDS 570
FINAL REPORT
MV PORTFOLIO OPTIMIZATION

Under the guidance of
Dr Pawel Kalczynski

Team T1

Duy Do
Juliet Cabral
Carolina Canas
Prateeksha Mehta
Preet Sanandiya

Table of Contents

Executive SUMMARY.....	3
Introduction: Project Framework and Objectives.....	4
Data Architecture ETL in PostgreSQL	6
The Strategic Role of the Custom Trading Calendar	6
Structuring and Populating End-of-Day (EOD) Data Tables	7
Establishing a Secure R-to-Database Connection.....	7
Data Preparation and Analysis in R	8
Data Retrieval and Initial Transformation	8
Advanced Data Conditioning for Time-Series Analysis.....	8
Deriving and Sanitizing Daily Returns	9
Mean-Variance Optimization Modeling	10
Exploratory Analysis of Training Data (2016-2020).....	10
Constructing the Optimization Framework.....	11
Deriving Optimal Portfolio Weights.....	12
Out-of-Sample Performance Evaluation (2021).....	14
Quantitative Performance Metrics	14
Visualizing Comparative Performance	15
Synthesizing the Performance Verdict.....	16
Conclusion.....	17
APPENDICES	19
Appendix A: SQL Code for Creating and Updating Tables	19
Appendix B: End of Day (EOD) Quotes and Indices in SQL.....	20
Appendix D: Data Retrieval, Preparation, and Calculating Returns.....	21
Appendix E: Portfolio Optimization	23

Executive Summary

This report presents a complete, end-to-end workflow for constructing and evaluating an equity portfolio using the Mean-Variance optimization framework. The project integrates a custom-built financial data architecture in PostgreSQL with advanced analytical processing in R to develop an optimized portfolio based on five years of historical daily return data (2016–2020). The portfolio's performance is subsequently assessed on out-of-sample data from the first quarter of 2021 to determine the robustness and practical viability of the optimization strategy.

A comprehensive Extract, Transform, Load (ETL) infrastructure was engineered to ensure the accuracy and continuity of all financial time series used in this study. This included the creation of a custom trading calendar, the consolidation of equity and benchmark datasets, and the implementation of a secure, read-only connection between the database and the analytical environment. Data were then cleaned, validated for completeness, converted into a structured time-series format, and transformed into sanitized return matrices suitable for quantitative modeling.

The Mean-Variance optimization model was constructed under realistic investment constraints, including a full-investment requirement and a minimum acceptable return (MAR) derived from the S&P 500 Total Return Index (SP500TR). Optimal weights were generated by minimizing portfolio variance while meeting the return constraint, resulting in a theoretically efficient allocation based on historical performance characteristics.

However, out-of-sample performance results reveal critical limitations. When applied to 2021 data, the optimized portfolio significantly underperformed the market benchmark across all key metrics, including annualized return and Sharpe ratio, despite exhibiting comparable risk levels. The portfolio's negative cumulative returns during most of the test window stand in sharp contrast to the benchmark's strong upward trajectory.

These findings underscore an important insight: while Mean-Variance optimization is mathematically elegant and operationally transparent, reliance on historical data can produce portfolios that are vulnerable to structural market shifts such as those induced by the COVID-19 pandemic, rendering past performance a poor predictor of future outcomes. The study highlights the need for enhancements to traditional optimization frameworks, including dynamic risk models, alternative covariance estimators, and scenario-based constraints that better reflect evolving market conditions.

Overall, this project demonstrates both the practical feasibility and inherent limitations of classical Mean-Variance portfolio construction. The documented workflow provides a replicable foundation for quantitative asset allocation while motivating more robust and adaptive approaches for real-world investment decision-making.

Introduction:

Project Framework and Objectives

This report details a complete and replicable workflow for constructing and evaluating an investment portfolio using the Mean-Variance optimization method. The methodology presented herein provides a structured approach, beginning with robust data engineering in a relational database, proceeding to quantitative modeling in a statistical programming environment, and culminating in a rigorous performance evaluation against a market benchmark.

The project's core objective is to construct an optimized portfolio using five years of historical daily return data, spanning from 2016 to 2020. The efficacy of this portfolio is then tested by evaluating its performance against out-of-sample data from the first three months of 2021 (January 1, 2021 – March 26, 2021).

The investment universe for this analysis consists of fifteen specific equities, identified by the following tickers:

Table 1 : Selected Tickers

Name of the Team Member	Ticker	Name of the Company
Duy Do	SSRM	SSR Mining Inc. Common Stock
	CMTV	Community Bancorp.
	GHM	Graham Corporation Common Stock
Cabral, Juliet	TTC	Toro Company (The) Common Stock
	HOPE	Hope Bancorp Inc. Common Stock
	LPL	LG Display Co Ltd AMERICAN DEPOSITORY SHARES
Canas, Carolina	PFE	Pfizer Inc. Common Stock
	IHG	Intercontinental Hotels Group American Depositary Shares
	CSL	Carlisle Companies Incorporated Common Stock
Mehta, Prateeksha	SATS	EchoStar Corporation Common Stock
	SNX	TD SYNEX Corporation Common Stock
	MTRX	Matrix Service Company Common Stock
Sanandiya, Preet	NGG	National Grid Transco PLC National Grid PLC
	ST	Sensata Technologies Holding plc Ordinary Shares
	MCO	Moody's Corporation Common Stock

Performance is benchmarked against the S&P 500 Total Return Index (SP500TR). The primary optimization goal is to minimize portfolio risk, defined as the standard deviation of returns. This objective is constrained by the requirement to achieve a minimum acceptable return (MAR), which is derived from the benchmark's performance during the five-year training period.

This report will systematically guide the reader through the data engineering, modeling, and evaluation phases of the project. By providing a transparent and practical methodology, it serves as a comprehensive guide for quantitative analysts seeking to implement a foundational portfolio optimization strategy.

Data Architecture ETL in PostgreSQL

A formalized Extract, Transform, Load (ETL) process is foundational to quantitative analysis, as it ensures the temporal integrity and statistical reliability of the input time series. The strategic importance of this phase cannot be overstated; a well-structured database foundation is critical for ensuring the data integrity and consistency that directly impact the reliability of any subsequent modeling. This section details the creation of a dedicated data architecture within a PostgreSQL environment to support the optimization workflow.

The Strategic Role of the Custom Trading Calendar

A custom trading calendar was established to serve as the chronological backbone for the entire analysis. A dedicated `custom_calendar` table was created in the stockmarket PostgreSQL database, spanning the full project date range from 2016 to 2021.

The table's initial schema consisted of six attributes designed to provide granular temporal context:

- **date**: The primary key for the table.
- **year (y)**: The calendar year.
- **month (m)**: The calendar month.
- **day (d)**: The day of the month.
- **day of week (dow)**: The day of the week.
- **trading**: A binary flag, with 1 indicating a trading day and 0 otherwise.

Two critical transformations were subsequently applied to this base calendar to enhance its analytical utility. First, a `prev_trading_day` attribute was added via a subquery, allowing for efficient lookups of the immediately preceding trading session for any given date. Second, an `eom` (end of month) attribute was incorporated using a left-sided self-join on the year, month, and `lastd`, which was the maximum trading day when the table was grouped by month. This flags the final trading day of each month—a common requirement for periodic performance reporting.

Structuring and Populating End-of-Day (EOD) Data Tables

With the calendar established, dedicated tables were created to house the end-of-day (EOD) market data.

The `eod_quotes` table was designed to store equity pricing data. It contains eight attributes and is indexed by a composite primary key of ticker and date to ensure a unique record for each security on each day.

The `eod_indices` table was structured identically to store benchmark data, also containing eight attributes and using a composite primary key of symbol and date.

The data loading process required careful consolidation. Because the analysis extended into 2021, two separate CSV files containing SP500TR data were merged into a single `SP500TR_Project` file. This consolidated file, along with the equity quote data, was then imported into the respective database tables, ensuring a complete and continuous time series for the entire project period.

Establishing a Secure R-to-Database Connection

To maintain principles of data governance and security, a dedicated, read-only database role was created specifically for this analytical task. The `stockmarketreader` role was established in PostgreSQL and granted `SELECT` permissions on all current and future tables within the `public` schema. This practice prevents accidental data modification from the analysis environment and aligns with the principle of least privilege.

The connection between the R Studio environment and the PostgreSQL database was established using the `RPostgres` package. The `dbConnect` function was used to create the connection object, which requires specifying several key parameters: the database user role (`stockmarketreader`), its associated password, the host, port, and the database name (`stockmarket`).

With the data architecture fully engineered and a secure connection pipeline established, the workflow transitions to the analytical environment for data preparation and feature engineering.

Data Preparation and Analysis in R

After establishing a secure database connection, the workflow shifted from data storage to data preparation. This phase, conducted entirely within R Studio, focused on retrieving the raw EOD data, performing essential cleaning and transformations, and preparing a sanitized dataset suitable for rigorous quantitative modeling.

Data Retrieval and Initial Transformation

From R, we created a query that retrieved the entire `custom_calendar` table and joined `eod_indices` and `eod_quotes` records for the specified time range, which is from December 31, 2015, to March 26, 2021. After retrieving the data, we prepared the calendar data (`ccal`) by filtering for trading days (`trading == 1`) and labeled this subset as `tdays`. The merged end-of-day (EOD) table was filtered for completeness by creating a list of symbols with at least 99% filled records and then using this list as a subset to create the `EOD_complete` data.

Advanced Data Conditioning for Time-Series Analysis

Before returns could be calculated, the raw price data required several conditioning steps to ensure its quality and structure were appropriate for time-series analysis.

First, a data completeness filter was applied. A list of symbols was generated containing only those securities that had at least 99% of their expected records present for the analysis period. This list was then used to subset the EOD data, effectively removing assets with insufficient historical data.

Next, the data was reshaped from a long format to a wide format using the `dcast` function from the `reshape2` package. This created a pivot table (`eod_pvt`) where dates served as rows, symbols became columns, and the values were the daily average adjusted close. This structure is standard for multivariate time-series analysis.

Final transformations were applied to this pivot table. The date column was set as the table's row names, a common convention in R for time-series objects, and the original date column was then removed. To handle any remaining null values, the Last Observation Carried Forward (LOCF) method was implemented using the `na.locf` function from the `zoo` package. This technique fills missing data points with the last known value, a standard approach for financial time series.

Deriving and Sanitizing Daily Returns

With a clean, complete, and properly structured price series, the daily returns were calculated. This was accomplished using the `CalculateReturns` function from the `PerformanceAnalytics` package, which efficiently computes the period-over-period percentage change for each asset. The resulting data frame of daily returns was named `eod_ret`.

Following the return calculation, the first row of the data frame (corresponding to December 31, 2015) contained NA values and was therefore removed.

Finally, as a data sanitization measure, a vectorized function (`colmax`) was implemented to identify the maximum daily return for each asset. Daily returns exceeding 1.0 (a 100% gain in a single day) were defined as extreme and potentially erroneous values. Only symbols whose maximum daily return was 1.0 or less were retained for the final analysis, ensuring that outliers did not unduly skew the optimization model.

With the data now fully cleaned and transformed into a sanitized matrix of daily returns, it was ready for core portfolio optimization modeling.

Mean-Variance Optimization Modeling

This section details the core analytical process of the project, moving from exploratory analysis of the historical training data to the construction of the Mean-Variance optimization model. The primary output of this phase is the set of optimal portfolio asset weights designed to minimize risk while achieving a predefined return target.

Exploratory Analysis of Training Data (2016-2020)

To optimize a portfolio, our team created a cumulative returns chart with the assigned tickers. A vector was created with the fifteen selected symbols, which we used as a subset for `eod_ret`. We turned the filtered data into an extensible time series (xts) that will be used for optimization. The xts with the ticker subset is labelled `Ra`, and to compare, we created a benchmark using SP500TR data, which is labelled `Rb`. We split the data into a training and testing set, with the period from 2016 to 2020 serving as the training range and 2021 as the test range. By using the function `chart.CumReturns` on the combined training set for `Ra` and `Rb`, we were able to create a chart with cumulative returns for each selected ticker and SP500TR.

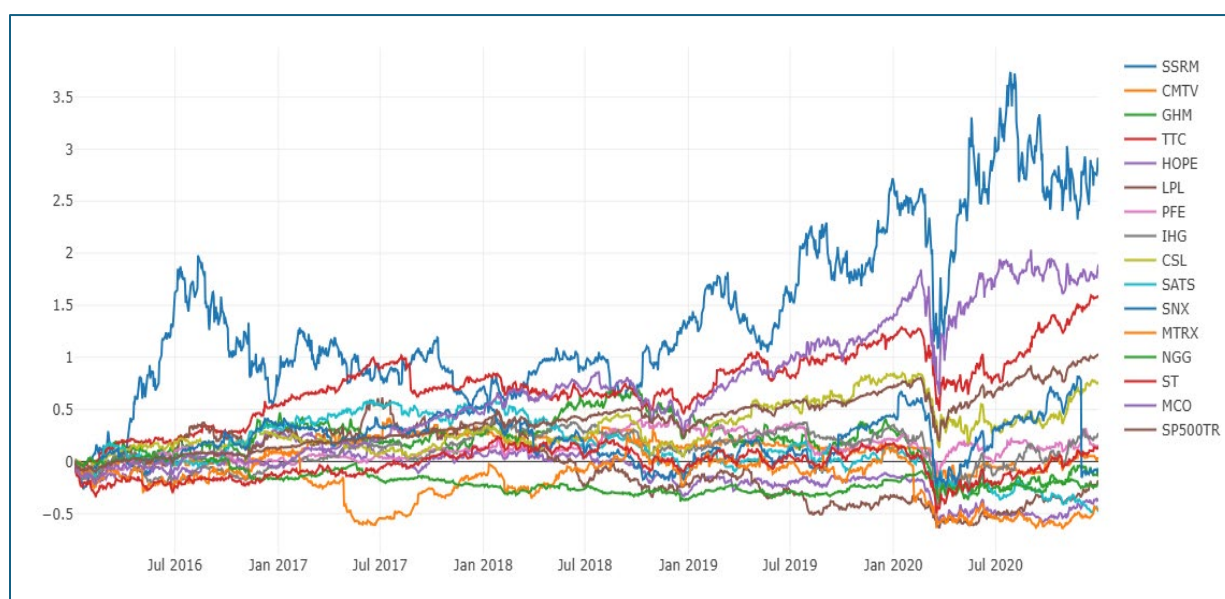


Figure X. Cumulative returns for selected equities and the SP500TR benchmark over the training period (2016–2020). Returns were computed from adjusted closing prices and plotted using the `chart.CumReturns` function in R.

As demonstrated by the graph, most tickers follow a seemingly stationary trend, while some show a positive trend over time. Notably, SSRM, a mining company, has noticeably higher returns than all others, with a major spike around mid-2016. There is a significant drop in returns for all tickers in the first quarter of 2020. The drop is likely due to the COVID-19 outbreak, which had a significant economic impact during this period. Now that we have investigated cumulative returns, we can optimize a portfolio using the selected tickers.

Constructing the Optimization Framework

To optimize a Mean-Variance portfolio, we need three additional packages: PortfolioAnalytics, ROI, and ROI.plugin.quadprog. First, we focused on the benchmark, SP500TR, data, and created a table for its annualized returns based on its training set.

SP500TR	
Annualized Return	0.1523
Annualized Std Dev	0.1923
Annualized Sharpe (Rf=0%)	0.7918

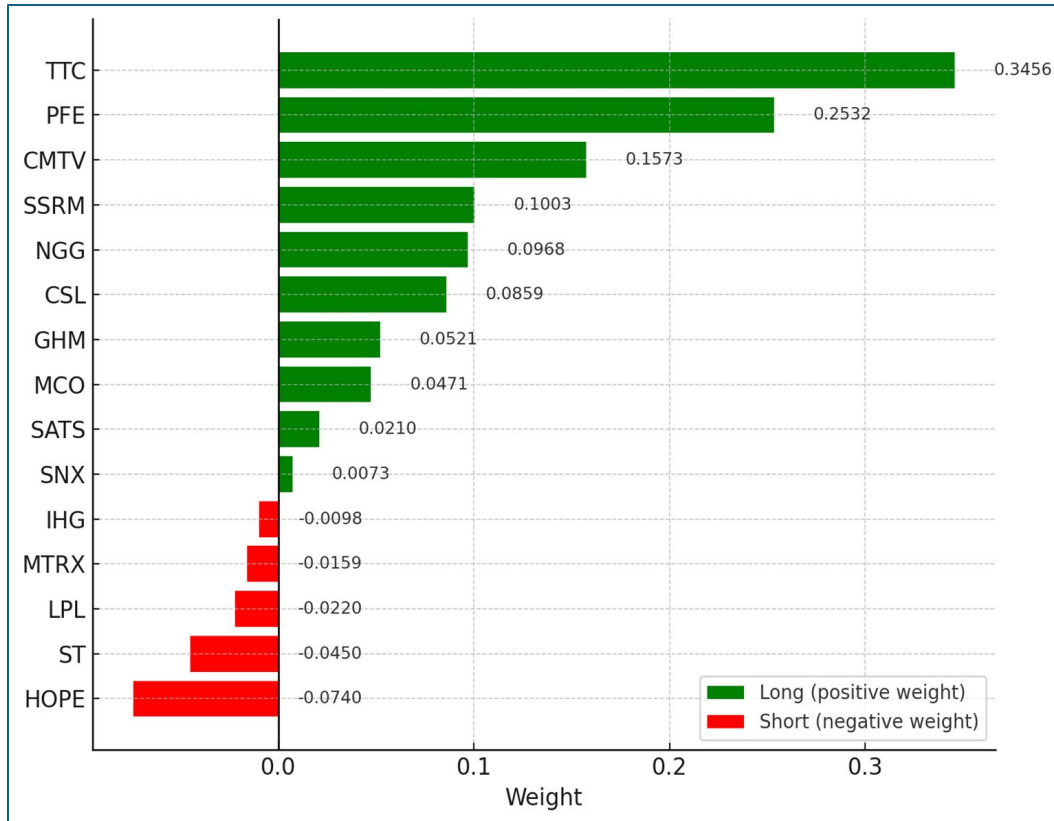
We then set the return target (MAR) as the average of the SP500TR daily returns and the investment universe (pspec) as the tickers in Ra. After, we specified the objective and two constraints. The objective is to minimize the risk and standard deviation using the specified tickers as the portfolio. One of the constraints is that the sum of the weights must be 1, indicating a fully invested type. The other constraint is that the optimization must achieve the minimum return target, which is MAR in this case.

Deriving Optimal Portfolio Weights

Using the `optimize.portfolio` function, with `Ra_training` as the asset, `pspec` as the portfolio, and `ROI` as the optimization method, we were able to create a portfolio. From the portfolio, we extracted the weights for each asset and calculated the sum of the weights, which equaled 1, meaning that the constraints were met.

Table 2: Optimal Weights of the selected stocks

Ticker	Weight
TTC	0.3456
PFE	0.2532
CMTV	0.1573
SSRM	0.1003
NGG	0.0968
CSL	0.0859
GHM	0.0521
MCO	0.0471
SATS	0.021
SNX	0.0073
IHG	-0.0098
MTRX	-0.0159
LPL	-0.022
ST	-0.045
HOPE	-0.074



After retrieving the weights from the training set, we applied them to the testing set (2021) by multiplying the testing set, $Ra_{testing}$, by the optimal weights found, opt_w . We created the data frame Rp , which includes $Rb_{testing}$ and ptf , representing the portfolio's performance as determined in the previous step. With the data frame, we can analyze the portfolio's performance based on its annualized and cumulative returns.

Out-of-Sample Performance Evaluation (2021)

The true test of any investment model lies in its performance on unseen data. This section assesses the real-world viability of the optimized portfolio by applying the calculated weights to the 2021 test data (January 4, 2021 – March 26, 2021) and comparing its performance against the SP500TR benchmark over the same period.

Quantitative Performance Metrics

To analyze the performance of the MV portfolio, we looked at the annualized returns of the portfolio and the benchmark, SP500TR.

Table 3 : Annualized Returns

	SP500TR	ptf
Annualized Return	0.2987	0.0191
Annualized Std Dev	0.1623	0.1515
Annualized Sharpe (Rf=0%)	1.8399	0.1259

Based on the returns shown on the chart, the portfolio's performance is significantly worse than the benchmark, as its return is nearly 28% lower than the SP500TR's return. Additionally, the Sharpe ratio, which is the return adjusted for risk, is significantly lower for the portfolio than for the benchmark. The standard deviations for both the portfolio and benchmark are nearly the same, indicating that the models' variances are comparable. Both the annualized return and the Sharpe ratio demonstrate that the portfolio created is a weak investment compared to the market during 2021. This takeaway is also clear in the cumulative returns chart for the portfolio and benchmark.

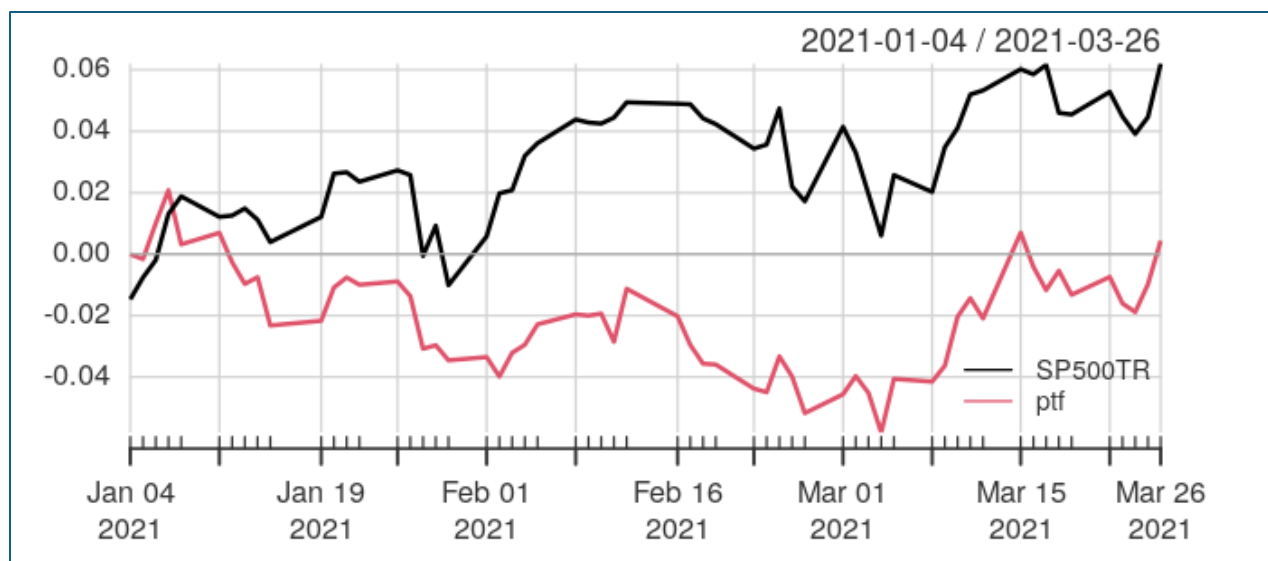


Chart 2: Predictive Cumulative Returns

The graph highlights the disparity between the returns of the portfolio and SP500TR. At the beginning of 2021, both the portfolio and benchmark started with similar positive cumulative returns; however, the portfolio's returns began to decline while the benchmark's continued to rise. For most of the first three months of 2021, the portfolio's cumulative returns were negative, with small positive spikes in early January, March, and on the last recorded date, March 26. On the other hand, the cumulative returns for the SP500TR were mainly positive during this timeframe, with a slight negative dip in late January. The benchmark was also following a positive trend on the last date recorded, but its cumulative return was significantly higher at this point than that of the portfolios.

Visualizing Comparative Performance

A cumulative returns chart for the test period provides a clear visual narrative of the portfolio's performance relative to the benchmark.

The chart shows that while both the portfolio and the SP500TR started at similar levels at the beginning of January 2021, their paths quickly diverged. The portfolio's cumulative returns declined almost immediately and remained negative for the vast majority of the quarter. In stark contrast, the benchmark's returns were predominantly positive and followed a clear upward trend throughout the period. This visual evidence powerfully corroborates the story told by the quantitative metrics.

Synthesizing the Performance Verdict

Based on a comprehensive review of both the quantitative and visual analyses, the conclusion is definitive. The Mean-Variance optimized portfolio constructed from 2016-2020 data was a weak investment compared to the market during the first quarter of 2021. Despite being optimized to minimize risk, it failed to deliver competitive returns, both on an absolute and risk-adjusted basis. The finding of poor out-of-sample performance naturally leads to a critical reflection on the methodology's assumptions and limitations.

Conclusion

This project created a detailed, database-driven workflow for building and assessing an equity portfolio using the Mean-Variance optimization (MVO) framework. By combining a robust ETL setup in PostgreSQL with structured data preparation and quantitative modeling in R, the study demonstrated how to transform historical financial data into a structured analysis environment that facilitates portfolio optimization under real-world investment conditions.

The optimized portfolio, based on daily returns from 2016 to 2020, met all model requirements. This included full capital allocation and achieving the minimum acceptable return (MAR) based on the S&P 500 Total Return Index. The optimization process worked as expected: asset weights were calculated to minimize portfolio variance while reaching a set return threshold, and no constraints were violated. The model thus operated correctly within the assumptions of the classical MVO framework.

However, the true test of any investment strategy is its performance outside the sample period. This is where the limits of Mean-Variance optimization became clear. When the trained portfolio weights were applied to the first quarter of 2021, the resulting portfolio significantly underperformed the benchmark. While the portfolio's volatility was similar to that of the S&P 500 TR, it generated significantly lower returns, resulting in negative cumulative performance for much of the period. The large gap between expected and actual results highlights the weaknesses of optimization methods that depend solely on historical return distributions.

This performance issue reflects a broader criticism of Mean-Variance optimization: the method assumes that statistical relationships remain stable over time, particularly regarding asset returns and covariances. Market conditions during and after the COVID-19 pandemic disrupted these assumptions, causing structural breaks, nonlinear volatility patterns, and return anomalies that were absent in the training data. As a result, historical mean returns and covariance matrices, which are key inputs for the MVO process, failed to consider future market behavior, leading to a portfolio optimized for a situation that no longer existed.

The findings from this study suggest several ways to improve methodology. Future work could include:

- Robust or shrinkage-based covariance estimators to lessen sensitivity to outliers,
- Regime-switching or dynamic optimization models that adjust to changing market conditions,
- Factor-based or fundamental constraints to limit exposure to individual assets or sectors, and
- Scenario-based stress testing to prepare for external shocks like those caused by the pandemic.

In summary, while this project demonstrates the operational success of Mean-Variance optimization and provides a complete and replicable workflow for portfolio construction, it also reveals the framework's weaknesses in rapidly changing market situations. The study emphasizes an important lesson for quantitative finance practitioners: optimization must move beyond historical averages and static assumptions to remain relevant in today's complex, data-rich, and highly volatile financial markets.

APPENDICES

Appendix A: SQL Code for Creating and Updating Tables

```
...
-- Create custom_calendar table
CREATE TABLE public.custom_calendar (
    date          DATE NOT NULL,
    y             INTEGER,
    m             INTEGER,
    d             INTEGER,
    dow           VARCHAR(3) COLLATE pg_catalog."default",
    trading       SMALLINT,
    CONSTRAINT custom_calendar_pkey PRIMARY KEY (date)
)
WITH (
    OIDS = FALSE
)
TABLESPACE pg_default;

-- Set table owner
ALTER TABLE public.custom_calendar
    OWNER TO postgres;

-- Clear custom_calendar table
DELETE FROM custom_calendar;

-- Import data from Excel to custom_calendar
SELECT * FROM custom_calendar order by date desc LIMIT 10;

-- Add extra columns
ALTER TABLE public.custom_calendar
    ADD COLUMN eom SMALLINT;

ALTER TABLE public.custom_calendar
    ADD COLUMN prev_trading_day DATE;

-- Update the table with prev_trading_day
UPDATE custom_calendar
SET prev_trading_day = PTD.ptd
FROM (SELECT date, (SELECT MAX(CC.date) FROM custom_calendar CC WHERE CC.trading=1
AND CC.date<custom_calendar.date) ptd FROM custom_calendar) PTD
WHERE custom_calendar.date = PTD.date;
-- CHECK
SELECT * FROM custom_calendar ORDER BY date;

-- Update the table with the end of the month
UPDATE custom_calendar
SET eom = EOMI.endofm
FROM (
```

```

SELECT CC.date,CASE WHEN EOM.y IS NULL THEN 0 ELSE 1 END endofm FROM
custom_calendar CC
LEFT JOIN
(SELECT y,m,MAX(d) lastd FROM custom_calendar WHERE trading=1 GROUP by y,m) EOM
ON CC.y=EOM.y AND CC.m=EOM.m AND CC.d=EOM.lastd
) EOMI
WHERE custom_calendar.date = EOMI.date;
-- CHECK
SELECT * FROM custom_calendar ORDER BY date;

```

Appendix B: End of Day (EOD) Quotes and Indices in SQL

```

...
-- Create table eod_quotes
CREATE TABLE public.eod_quotes
(
    ticker character varying(16) COLLATE pg_catalog."default" NOT NULL,
    date date NOT NULL,
    adj_open real,
    adj_high real,
    adj_low real,
    adj_close real,
    adj_volume numeric,
    CONSTRAINT eod_quotes_pkey PRIMARY KEY (ticker, date)
)
WITH (
    OIDS = FALSE
)
TABLESPACE pg_default;

ALTER TABLE public.eod_quotes
    OWNER to postgres;

-- Import the CSV file eod_quotes
-----
SELECT ticker,date,adj_close FROM eod_quotes WHERE date BETWEEN '2015-12-31' AND '2021-
12-31' ORDER BY DATE; --8989 rows
SELECT ticker,date,adj_close FROM eod_quotes WHERE date BETWEEN '2014-12-31' AND '2020-
12-31' ORDER BY DATE; --10251 rows

-- Create eod_indices table
CREATE TABLE public.eod_indices (
    symbol    VARCHAR(16) COLLATE pg_catalog."default" NOT NULL,
    date      DATE NOT NULL,
    open      REAL,
    high      REAL,
    low       REAL,
    close     REAL,
    adj_close REAL,
    volume    DOUBLE PRECISION,
    CONSTRAINT eod_indices_pkey PRIMARY KEY (symbol, date)

```

```

)
WITH (
  OIDS = FALSE
)
TABLESPACE pg_default;

-- Set table owner
ALTER TABLE public.eod_indices
  OWNER TO postgres;

-- Clear current table eod_indices
DELETE FROM eod_indices;

-- Import CSV file: SP500TR_Project.csv
-- Check date range
SELECT MIN(DATE), MAX(DATE) FROM eod_indices;

```

Appendix C: Reader Role in SQL and Connecting to R

```

...
-- rolename: stockmarketreader
DROP OWNED BY stockmarketreader;
DROP USER stockmarketreader;

CREATE USER stockmarketreader WITH
LOGIN
NOSUPERUSER
NOCREATEDB
NOCREATEROLE
INHERIT
NOREPLICATION
CONNECTION LIMIT -1
-- Grant read rights (on existing tables and views)
GRANT SELECT ON ALL TABLES IN SCHEMA public TO stockmarketreader;

-- Grant read rights (for future tables and views)
ALTER DEFAULT PRIVILEGES IN SCHEMA public
  GRANT SELECT ON TABLES TO stockmarketreader;

```

Appendix D: Data Retrieval, Preparation, and Calculating Returns

```

#-----
# Step 1: Connect DB to db stockmarket
#-----
require(RPostgres) # load driver
conn <- dbConnect(RPostgres::Postgres(),
  user="stockmarketreader",
  password="read123",
  host="postgres", #localhost if run on R Studio
  port=5432,
  dbname="stockmarket"
)

```

```

#-----
# Step 2: Data Retrieval
#-----
qry <- 'SELECT * FROM custom_calendar ORDER by date'
ccal <- dbGetQuery(conn, qry) #store select qry to ccal: custom_calendar

#union qry1 and qry2 into eod data frame
qry1="SELECT symbol,date,adj_close FROM eod_indices WHERE date BETWEEN '2015-12-31'
AND '2021-03-26'"
qry2="SELECT ticker,date,adj_close FROM eod_quotes WHERE date BETWEEN '2015-12-31' AND
'2021-03-26'"
eod <- dbGetQuery(conn, paste(qry1,'UNION',qry2)) #store select qry1+qry2 to eod

#Disconnect and remove connection
dbDisconnect(conn);
rm(conn)

#-----
# Step 3: Data Preparation
#-----
#Export the Trading day using DB ccal where trading = 1
tdays<-ccal[which(ccal$trading==1),,drop=F]

#Check Completeness: Ensure each symbol has ≥99% of trading days
pct<-table(eod$symbol)/(nrow(tdays)-1);
selected_symbols_daily<-names(pct)[which(pct>=0.99)];

#Store all the symbols which belong good list (99%) from eod to eod_complete
eod_complete<-eod[which(eod$symbol %in% selected_symbols_daily),,drop=F]

#Convert data from eod_complete to pivot table
require(reshape2)
eod_pvt<-dcast(eod_complete, date ~ symbol,
               value.var='adj_close',fun.aggregate = mean, fill=NULL)

#Merging tdays and eod_pvt
eod_pvt_complete <-
  merge.data.frame(
    x = tdays[, 'date', drop = FALSE],
    y = eod_pvt,
    by = 'date',
    all.x = TRUE
  )

#Change rowname to date
rownames(eod_pvt_complete)<-eod_pvt_complete$date

#Remove date column from eod_pvt_complete
eod_pvt_complete$date<-NULL;

#Replace a few missing (NA or NaN) with previous data

```

```

# using LOCF: Last Observation Carried Forward
require(zoo)
eod_pvt_complete <- na.locf(
  eod_pvt_complete,
  na.rm = FALSE,
  fromLast = FALSE,
  maxgap = 3
)
#-----
# Step 4: Calculating Returns
#-----
#load package PerformanceAnalytics and calc Return to eod_ret
require(PerformanceAnalytics)
eod_ret<-CalculateReturns(eod_pvt_complete)

#remove the first row with NA
eod_ret<-tail(eod_ret,-1)

#Extreme Returns Check
#create function colMax : return max return in column
colMax <- function(data) sapply(data, max, na.rm = TRUE)

#Calculate max daily return in eod_ret
max_daily_ret<-colMax(eod_ret)

#Check completeness
selected_symbols_daily<-names(max_daily_ret)[which(max_daily_ret<=1.00)]

#subset eod_ret with selected_symbols_daily
eod_ret<-eod_ret[,which(colnames(eod_ret) %in% selected_symbols_daily),drop=F]

#Export eod_ret.csv
write.csv(eod_ret,'eod_ret.csv')

```

Appendix E: Portfolio Optimization

```

#-----
# Step 5: Optimizing Portfolio
#-----
#Store return of assigned ticker to Ra
tickers <- c(
  'SSRM','CMTV','GHM', #Do, Duy
  'TTC','HOPE','LPL', #Cabral, Juliet
  'PFE','IHG','CSL', #Canas, Carolina
  'SATS','SNX','MTRX', #Mehta, Prateeksha
  'NGG','ST','MCO' #Sanandiya, Preet
)
Ra <- as.xts(eod_ret[, tickers, drop = FALSE])

#Store return of SP500TR to Rb
Rb <- as.xts(eod_ret[, 'SP500TR', drop = FALSE])

```

```

#Training: 2016-01-01 to 2020-12-31
Ra_training <- Ra["2016-01-01/2020-12-31"]
Rb_training <- Rb["2016-01-01/2020-12-31"]

#Test: 2021-01-01 to 2021-03-26
Ra_testing <- Ra["2021-01-01/2021-03-26"]
Rb_testing <- Rb["2021-01-01/2021-03-26"]

#Cumulative returns chart
chart.CumReturns(cbind(Ra_training, Rb_training), legend.loc = 'topleft')

#Optimize the MV (Markowitz 1950s) portfolio weights based on training
#Desc: Find me a portfolio using these assets,
#   where all the money is invested (weights sum to 1),
#   the expected return is at least mar, and among all such portfolios,
#   minimize the standard deviation (risk)

#Load packages needed
require(PortfolioAnalytics);
require(ROI);
require(ROI.plugin.quadprog)

#Benchmark summary
table.AnnualizedReturns(Rb_training)

#Set the return target (MAR)
mar<-mean(Rb_training)

#Specify the asset universe
pspec<-portfolio.spec(assets=colnames(Ra_training))

#Objective: minimize risk, minimize portfolio standard deviation
pspec<-add.objective(portfolio=pspec,type="risk",name='StdDev')

#Constraint: fully invested, Sum of weights = 1
pspec<-add.constraint(portfolio=pspec,type="full_investment")

#Constraint: minimum return, Enforces expected portfolio return  $\geq$  mar
pspec<-add.constraint(portfolio=pspec,type="return",return_target=mar)

#Solve: optimize portfolio
opt_p<-optimize.portfolio(R=Ra_training,portfolio=pspec,optimize_method = 'ROI')

#extract weights (negative weights means shorting)
opt_w<-opt_p$weights

#sum of weights
sum <- sum(opt_w)

#apply weights to test returns

```



```
Rp<-Rb_testing
```

```
#Taking optimized weights & applying them to the out-of-sample returns (Ra_testing)
```

```
#to compute the portfolio's performance during the test window
```

```
Rp$ptf<-Ra_testing %*% opt_w
```

```
#Check
```

```
head(Rp); tail(Rp)
```

```
#Showing chart compare SP500 with Portforlio
```

```
table.AnnualizedReturns(Rp)
```

```
chart.CumReturns(Rp, legend.loc = 'bottomright')
```

```
#Showing Weights
```

```
View(opt_w)
```